RGB-D Tomato Mass Estimation — Three Models Comparison

Purpose: technical reference for an LLM assisting with the design
of a clean, principled mass-head architecture.

Author context:
  - Master's thesis project, multitask RGB-D detection + mass regression
  - Architecture family: Detectron2 Mask R-CNN + custom mass head
  - Backbone: ResNet-50 + FPN, modified stem.conv1 to accept 4 channels
    (channels 0-2: BGR, channel 3: raw depth in mm)
  - Target: per-instance cluster mass regression in log-space
    (predict log(mass_kg); loss = MSE(pred_log_mass, log(gt_mass)))

Document scope:
  1. Architecture of each of three trained models
  2. Training configuration and validation metrics
  3. Test-set inference results (150 images, 7 unique mass values)
  4. Training data distribution (875 images, 36 unique mass values)
  5. Cross-model comparison and observed failure modes
  6. Conclusions relevant to designing a new mass head

All numbers in this document are taken from:
  - output/<model>/metrics.json
  - detectron2_no_late_fusion/results_16_5.txt
  - git history (commits d6fc3a0 and 5e3a6f7)
  - bash command history
No values are estimated or hypothetical unless explicitly stated.

Model 1: mass_stage4 — Architecture

```
Trained: 2026-02-01
Script:  train.py
Architecture commit: d6fc3a0 ('Clean start')

Meta-architecture:
  RGBDGeneralizedRCNN (subclass of GeneralizedRCNN)
  - _cache_depth(): caches normalized depth tensor on roi_heads
    (depth = (raw_mm - 915.043) / 411.683, invalid pixels masked to 0)
  - Both forward() and inference() call _cache_depth() before parent forward

Detection branch (standard Mask R-CNN, 4-channel input):
  - Backbone: ResNet-50 + FPN, conv1 expanded to 4 input channels
  - RPN, Box head, Mask head: unchanged from Detectron2 defaults
  - PIXEL_MEAN  = [103.530, 116.280, 123.675, 915.043]
  - PIXEL_STD   = [1.0,     1.0,     1.0,     411.683]
  - NUM_CLASSES = 1 (tomato_bunch)
  - SCORE_THRESH_TEST = 0.7
  - ROI_HEADS.BATCH_SIZE_PER_IMAGE = 512
  - BACKBONE.FREEZE_AT = 0

Late fusion (ACTIVE for this model):
  - For each FPN level (P2..P5) used by mass head:
      d_k = F.interpolate(cached_depth, feat_k.shape[-2:],
                          mode='bilinear', align_corners=False)
      out[k] = torch.cat([feat_k, d_k], dim=1)  # 256 -> 257 channels
  - NO scaling factor on depth (×8 scaling was added in commit 8796f8f,
    AFTER mass_stage4 was trained)

Mass head (MassRCNNConvUpsampleHead, d6fc3a0 version):
  Input: (N, 257, 14, 14)  # 256 FPN + 1 depth, after ROI pooling
    -> Conv3x3 + BN + ReLU   (257 -> 256)  # mass_fcn1
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn2
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn3
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn4
    -> ConvTranspose2d stride 2    (256 -> 256, 14x14 -> 28x28)
    -> AdaptiveAvgPool2d((4, 4))   (256 -> 256, 28x28 -> 4x4)
    -> Flatten                     (256*4*4 = 4096-d)
    -> Linear(4096 -> 128) + ReLU + Dropout(0.5)
    -> Linear(128 -> 1)
  Output: predicted log(mass_kg) per ROI

Mass loss:
  L_mass = w * MSE(pred_log_mass, log(gt_mass_kg))
  w = mass_loss_weight (config, see next page)
```

Model 1: mass_stage4 — Training Configuration

Inferred from bash history and metrics.json (LR confirmed in metrics):

```
  python train.py \
      --num_iterations 10000 \
      --learing_rate 0.00002 \        # 2e-5, constant
      --mass_loss_weight 2 \
      --weights ./output/<segm_tune-chain>/model_final.pth \
      --experiment_name 'mass_stage4' \
      --dataset_path ./data/
```

Notes:
- Trained as the final stage of a staged training chain:
  segm_tune (detection only) -> mass_stage3 -> mass_stage4
- Starting weights came from a detection-pretrained checkpoint
  (segm_tune family reached bbox/AP ~76; mass_stage3/4 chain
  fine-tuned with mass loss active)
- LR is constant at 2e-5 throughout (no warmup, no decay)
- IMS_PER_BATCH = 2 (default)
- Stratified sampler ON (mass bins: <0.25, 0.25-0.5, >=0.5 kg)
- Augmentation: multi-scale resize + horizontal flip + color jitter
  (color jitter on BGR channels only)

Solver:
- MAX_ITER   = 10000
- EVAL_PERIOD   = 500
- CHECKPOINT_PERIOD = 500
- No LR schedule (constant)

Initial checkpoint state at iter 499 (from metrics.json):
- bbox/AP = 91.6   (already very high -> detection was pretrained)
- segm/AP = 98.6
- loss_mass (train) ~ 0.17
- val_mass_loss ~ 0.157

This is the staged-training distinguishing feature: when mass_stage4
begins, detection is already converged, so the mass head is the only
component doing meaningful learning.

Model 1: mass_stage4 — Validation Metrics (from metrics.json)

10,000 iterations, 501 metric rows.
Eval period 500 iters, so 20 evaluations.

Train vs validation loss trajectory (loss = MSE on log_mass):

iter	train_loss_mass	val_loss_mass	gap	R^2	MAE
499	0.1445	0.1566	+0.012	+0.849	0.114
999	0.1239	0.1786	+0.055	+0.766	0.148
2999	0.0716	0.1531	+0.082	+0.878	0.104
4999	0.0548	0.1483	+0.094	+0.872	0.107
6999	0.0410	0.1413	+0.100	+0.883	0.103
8999	0.0330	0.1371	+0.104	+0.867	0.110
9499	0.0339	0.1367	+0.103	+0.889	0.100
9999	0.0330	0.1357	+0.103	n/a	n/a

Summary statistics:
- Train loss reduced 5.2x (0.172 -> 0.033)
- Val loss reduced 1.15x (0.157 -> 0.136)
- Max train/val ratio ~ 4.1x
- Peak val R^2 = 0.889 @ iter 9499
- Peak val MAE = 0.100 kg @ iter 9499
- bbox/AP stable around 91 (started high, did not improve)
- segm/AP stable around 98 (started high, did not improve)

Overfitting analysis:
- Mild train-val gap (~4x ratio at convergence)
- Low LR + stable detection backbone produced gradual, controlled learning of the mass head
- Compared to clean_baseline_v1_LF (~100x train-val ratio): mass_stage4 overfits ~25x less in the train/val-gap sense

Model 1: mass_stage4 — Test-set Inference Results

Test set: 150 images, 7 unique mass values, 3 imaging sessions
(Dec 2023, Mar 2025, Jul 2025).

Overall test summary:
  - MAE = 0.107 kg
  - MAPE = 19.4%
  - Mean signed error (bias) = -0.026 kg
  - Worst absolute error: 0.401 kg

Per-mass-group breakdown (test):

GT(kg)	n	mean_pred	MAE	MAPE	bias
0.108	21	0.117	0.013	12.3%	+0.009
0.165	14	0.127	0.038	22.8%	-0.038
0.296	22	0.269	0.040	13.7%	-0.027
0.430	15	0.433	0.030	6.9%	+0.003
0.600	39	0.740	0.140	23.4%	+0.140
0.765	15	0.448	0.317	41.5%	-0.317
0.947	24	0.793	0.154	16.3%	-0.154

Train-set inference (875 images, 36 unique mass values):
  - MAE = 0.020 kg
  - MAPE = 3.7%
  - Mean signed error (bias) = +0.008 kg
  - Per-mass-group MAPE: ranges 1.5% - 10.9% across all 36 values
  - All masses fit within ~5% on average

Observations:
  - Predicts physically grounded values for small clusters
    (0.108 -> 0.117 kg; near-zero bias)
  - Predictions follow GT for masses up to ~0.43 kg
  - Underestimates large clusters (0.765 -> 0.448; 0.947 -> 0.793)
  - High prediction variance on the 0.765 kg cluster specifically
    (individual predictions range 0.375 to 0.914)
  - Bias direction: positive for small, negative for large
    (opposite of regression-to-mean toward training mean of 0.548 kg)

Model 2: clean_baseline_v1 — Architecture (no late fusion)

```
Trained: 2026-04-15
Script:  train_new.py
Architecture state: current main, post-removal of late fusion
                    (committed in 5e3a6f7 'late fusion removed,
                    simplified mass head')

Meta-architecture:
  RGBDGeneralizedRCNN, but reduced to a no-op wrapper:
    class RGBDGeneralizedRCNN(GeneralizedRCNN): pass
  No _cache_depth, no late-fusion path.

Detection branch (standard Mask R-CNN, 4-channel input):
  - Same as Model 1 (ResNet-50 + FPN, 4-channel conv1)
  - Same PIXEL_MEAN/STD, NUM_CLASSES=1, BACKBONE.FREEZE_AT=0

Late fusion: NONE.
  Depth enters the network only via channel 3 of conv1.
  After conv1, depth information is implicit in 256-channel FPN features.

Mass head (simplified):
  Input: (N, 256, 14, 14)  # 256 FPN, after ROI pooling — NO depth concat
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn1
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn2
    -> Conv3x3 + BN + ReLU   (256 -> 256)  # mass_fcn3
    -> AdaptiveAvgPool2d((1, 1))   (256 -> 256, GAP)
    -> Flatten                     (256-d)
    -> Linear(256 -> 64) + ReLU + Dropout(0.5)
    -> Linear(64 -> 1)
  Output: predicted log(mass_kg) per ROI

Key differences from Model 1's mass head:
  - Input channels: 256 (no depth concat) vs 257 (with depth concat)
  - 3 conv layers vs 4 conv layers
  - No transposed conv (no spatial upsampling)
  - Pool to (1,1) instead of (4,4): destroys spatial info before MLP
  - MLP: 256->64->1 (~17k params) vs 4096->128->1 (~540k params)
  Total mass head capacity is substantially smaller.
```

Model 2: clean_baseline_v1 — Training Configuration

```
Exact command (from bash_history):

  python train_new.py \
      --num_iterations 15000 \
      --mass_loss_weight 1.0 \
      --rgb_dropout 0.0 \
      --experiment_name clean_baseline_v1 \
      --dataset_path ./data/

Resolved defaults (from train_new.py):
  --weights         = ./mask_rcnn_R50_FPN_3x_RGBD.pth  (base 4-channel ckpt)
  --batch_size      = 2 (IMS_PER_BATCH)
  --learning_rate   = 1e-4 (BASE_LR)
  --eval_period     = 500
  --checkpoint_period = 1000
  WARMUP_ITERS  = min(500, max_iter // 10) = 500
  WARMUP_FACTOR = 1/1000
  STEPS         = (12000,)  # 80% of MAX_ITER
  GAMMA         = 0.1       # x0.1 LR decay at iter 12000

Other config:
  - BACKBONE.FREEZE_AT = 0  (whole backbone trainable)
  - ROI_HEADS.BATCH_SIZE_PER_IMAGE = 512
  - SCORE_THRESH_TEST = 0.7
  - Stratified sampler ON (small/medium/large mass bins)
  - rgb_dropout = 0.0 (RGB channels never replaced)
  - Color jitter: brightness/contrast 0.6-1.4 on BGR (80% probability)
  - Multi-scale resize + horizontal flip on all 4 channels

LR trajectory observed in metrics.json:
  - iter 19:    LR = 3.9e-6  (warming up from 1e-7)
  - iter 3699:  LR = 1e-4    (full LR)
  - iter 14999: LR = 1e-5    (post-decay)

This is the canonical joint-from-scratch training configuration.
Detection, mask head, and mass head all learn simultaneously
from a base 4-channel pretrained checkpoint.
```

Model 2: clean_baseline_v1 — Validation Metrics

15,000 iterations, 1510 metric rows.
Eval period 500 iters.

Train vs validation loss trajectory:

iter	train_loss_mass	val_loss_mass	gap	R^2	MAE
499	0.082	0.293	+0.21	-18.27	0.156
2999	0.012	0.250	+0.24	+0.632	0.198
4999	0.009	0.218	+0.21	+0.690	0.178
6999	0.005	0.218	+0.21	+0.701	0.179
9999	0.003	0.207	+0.20	+0.594	0.210
11999	0.003	0.207	+0.20	+0.594	0.222
14999	0.002	0.202	+0.20	+0.544	0.222

Summary statistics:
- Train loss reduced 180x (0.38 -> 0.002)
- Val loss reduced 1.45x (0.293 -> 0.202)
- Train/val loss ratio at end: ~96x (clear overfit)
- Peak val R^2 = 0.594 @ iter 9999
- Peak val MAE = 0.210 kg @ iter 9999
- bbox/AP peak = 88.16 @ iter 2999 (stable after)
- segm/AP peak = 96.49 @ iter 2999
- Val MRE: bottoms 0.55 @ iter 3499, degrades to 0.77 @ end
  (regression-to-the-mean drift over training)

Overfitting analysis:
- Stable overfit (val loss plateaus, does not diverge)
- The simpler mass head (~17k MLP params) cannot fit training
  distribution tightly; train MAE on inference run was 0.108 kg (10%)
- Capacity ceiling reached before any clear test improvement

Model 2: clean_baseline_v1 — Test-set Inference Results

Test set: 150 images, 7 unique mass values.
Checkpoint: model_0009999.pth (peak R^2 iteration).

Overall test summary:
  - MAE = 0.204 kg
  - MAPE = 87.8%
  - Mean signed error (bias) = +0.175 kg
  - Worst absolute error: 0.550 kg

Per-mass-group breakdown (test):

GT(kg)	n	mean_pred	MAE	MAPE	bias
0.108	21	0.456	0.348	322.4%	+0.348
0.165	14	0.388	0.223	135.1%	+0.223
0.296	22	0.578	0.282	95.2%	+0.282
0.430	15	0.654	0.224	52.1%	+0.224
0.600	39	0.810	0.210	35.0%	+0.210
0.765	15	0.756	0.039	5.2%	-0.009
0.947	24	0.869	0.078	8.2%	-0.078

Train-set inference (20-image sample, NOT full train):
  - MAE = 0.108 kg (MAPE = 30.2%)
  - Predictions on training data are NOT tight (>10% error typical)
  - Indicates mass head reached its capacity ceiling on training data

Observations:
  - Strong positive bias across all clusters below 0.7 kg
  - Accurate around 0.7-0.95 kg (training-density region)
  - Classic regression-to-density: predictions pulled toward
    where training has the most examples
  - Mass head capacity (256->64->1 MLP) is insufficient to encode
    the full mass relationship; defaults to learned density prior

Model 3: clean_baseline_v1_LF — Architecture (late fusion restored)

```
Trained: 2026-05-03
Script:  train_new.py
Architecture state: late-fusion files restored from d6fc3a0,
                    explicitly without the x8 depth scaling factor.

Meta-architecture:
  RGBDGeneralizedRCNN with _cache_depth() and forward()/inference()
  overrides restored (same as Model 1)
  - The 4th-channel fallback (extract depth from image[3:4,:,:] at
    inference) is preserved, so train_new.py's mapper works without
    requiring a separate 'depth' key in batched_inputs.

Detection branch: same as Models 1 and 2 (no change).

Late fusion (ACTIVE, NO x8 scaling):
  Identical to Model 1:
    d_k = F.interpolate(cached_depth, feat_k.shape[-2:],
                        mode='bilinear', align_corners=False)
    out[k] = torch.cat([feat_k, d_k], dim=1)

Mass head:
  IDENTICAL to Model 1's mass head:
    257 -> 4x Conv3x3+BN+ReLU -> ConvTranspose2d -> AdaptiveAvgPool(4,4)
    -> Flatten -> Linear(4096->128) + ReLU + Dropout(0.5) -> Linear(128->1)
  Output: predicted log(mass_kg) per ROI

So Models 1 and 3 share THE SAME ARCHITECTURE.
Differences between Models 1 and 3 are training-related only.
```

Model 3: clean_baseline_v1_LF — Training Configuration

Exact command (from bash_history):

```
  python train_new.py \
      --num_iterations 15000 \
      --mass_loss_weight 1.0 \
      --rgb_dropout 0.0 \
      --experiment_name clean_baseline_v1_LF \
      --dataset_path ./data/
```

Resolved defaults: identical to Model 2
  - Same starting weights: ./mask_rcnn_R50_FPN_3x_RGBD.pth
  - Same BASE_LR = 1e-4 with warmup/decay
  - Same MAX_ITER = 15000
  - Same mass_loss_weight = 1.0 (constant from iter 0)
  - Same stratified sampler, augmentations, ROI batch size, etc.

Key delta from Model 2: the late-fusion architecture is active,
                        so the mass head now receives 257 channels
                        instead of 256.

Key deltas from Model 1 (mass_stage4):
  - Joint-from-scratch training (vs staged from segm_tune)
  - LR with warmup/decay (vs constant 2e-5)
  - mass_loss_weight = 1.0 (vs 2.0)
  - 15000 iters (vs 10000)
  - Higher peak LR (1e-4 vs 2e-5, 5x larger)

LR trajectory observed in metrics.json:
  - iter 19:   LR = 3.9e-6 (warming up)
  - iter 3699: LR = 1e-4 (full)
  - iter 14999: LR = 1e-5 (after decay at iter 12000)

Model 3: clean_baseline_v1_LF — Validation Metrics

15,000 iterations, 1501 metric rows. Eval period 500.

Train vs validation loss trajectory:

iter	train_loss_mass	val_loss_mass	gap	R^2	MAE
499	0.082	0.293	+0.21	-18.27	0.156
2999	0.012	0.250	+0.24	+0.632	0.198
5999	0.006	0.225	+0.22	+0.701	0.178
9499	0.003	0.207	+0.20	+0.757	0.162
11999	0.003	0.205	+0.20	+0.709	0.174
14999	0.002	0.202	+0.20	+0.737	0.168

Summary statistics:
- Train loss reduced 180x (0.38 -> 0.002)
- Val loss reduced 1.45x (0.293 -> 0.202)
- Train/val ratio at end: ~96x
- Peak val R^2 = 0.757 @ iter 9499
- Peak val MAE = 0.162 kg @ iter 9499
- bbox/AP peak = 89.65 @ iter 4499 (stable after)
- segm/AP peak = 97.42 @ iter 5499
- Val MRE: bottoms 0.405 @ iter 3999, ends 0.444
  (less drift than Model 2; mass head was more stable)

Comparison to Model 2 (same training, no late fusion):
- Same train-val gap ratio (~96x)
- Val R^2 +0.16 absolute improvement (0.594 -> 0.757)
- Val MAE -23% (0.210 -> 0.162)
- bbox/AP and segm/AP essentially unchanged
- MRE drift much milder

Conclusion: late fusion improves mass prediction within the validation distribution; it does NOT reduce the train/val gap.

Model 3: clean_baseline_v1_LF — Test-set Inference Results

Test set: 150 images, 7 unique mass values.
Checkpoint: model_0009999.pth (peak R^2 was at iter 9499; 9999 used here).

Overall test summary:
  - MAE = 0.127 kg
  - MAPE = 46.9%
  - Mean signed error (bias) = +0.121 kg
  - Worst absolute error: 0.389 kg

Per-mass-group breakdown (test):

GT(kg)	n	mean_pred	MAE	MAPE	bias
0.108	21	0.289	0.181	167.3%	+0.181
0.165	14	0.217	0.052	31.6%	+0.052
0.296	22	0.450	0.154	52.0%	+0.154
0.430	15	0.582	0.152	35.3%	+0.152
0.600	39	0.791	0.191	31.8%	+0.191
0.765	15	0.798	0.057	7.5%	+0.033
0.947	24	0.946	0.023	2.4%	-0.001

Train-set inference (20-image sample):
  - MAE = 0.058 kg, MAPE = 10.7%
  - Predictions on training data are tight (single-digit %
    error typical), confirming train-distribution memorization

Observations:
  - Same positive-bias-on-small-clusters pattern as Model 2,
    but with substantially smaller magnitudes
  - Strongest improvement vs Model 2 is on small clusters:
    0.108: 322% -> 167% MAPE
    0.165: 135% -> 32% MAPE
    0.296: 95% -> 52% MAPE
  - Excellent for clusters near training-density peak (>0.7 kg)
  - Bias direction is uniformly positive for clusters < 0.7 kg,
    consistent with regression-to-density of training distribution

Dataset: Training Distribution

```
Total: 875 images, 36 unique mass values.
Statistics:
  - Range: 46 g to 1010 g
  - Mean: 548 g (0.548 kg)
  - Median: ~560 g

Distribution by stratified-sampler bins (train.py: <250, 250-500, >=500 g):
  - Small (<250 g): 12 unique values, 155 images (17.7%)
  - Medium (250-500 g): 11 unique values, 262 images (30.0%)
  - Large (>=500 g): 13 unique values, 458 images (52.3%)

Most densely-sampled mass values (count >= 35):
    164 g:  43 imgs
    208 g:  45 imgs
    490 g:  45 imgs
    566 g:  35 imgs
    670 g:  46 imgs
    790 g:  50 imgs
    820 g:  58 imgs
    940 g:  54 imgs
   1010 g:  64 imgs

Sparsely-sampled mass values (count < 10):
     46 g:   6 imgs
     70 g:   8 imgs
     90 g:   6 imgs
     93 g:   6 imgs
    122 g:   5 imgs
    186 g:   9 imgs
    239 g:   9 imgs

Bias summary:
  - Distribution is heavily skewed toward larger clusters
  - 30.7% of all training images are in 790-1010 g range
  - Smallest clusters (<100 g) have <=10 images each
  - Stratified sampler equalizes BIN frequency but not WITHIN-bin diversity
```

Dataset: Test Distribution and Distribution Shift

```
Total: 150 images, 7 unique mass values.
Substantially smaller than train; almost certainly 7 distinct
physical clusters each photographed many times.

Test mass values and image counts:
  - 108 g:  21 imgs   (filename pattern: 2023-12-01-...)
  - 165 g:  14 imgs   (filename pattern: 2023-12-01-...)
  - 296 g:  22 imgs   (filename pattern: 2025-03-28-...)
  - 430 g:  15 imgs   (filename pattern: 2023-12-01-...)
  - 600 g:  39 imgs   (filename pattern: 2025-07-13-...)
  - 765 g:  15 imgs   (filename pattern: 2025-03-28-...)
  - 947 g:  24 imgs   (filename pattern: 2025-03-28-...)

Imaging sessions observed:
  - 2023-12-01: clusters of 108, 165, 430 g  (older, possibly different
                lighting, camera setup, season)
  - 2025-03-28: clusters of 296, 765, 947 g  (intermediate)
  - 2025-07-13: cluster of 600 g  (same date as observed in train images)

Distribution-shift considerations:
  - Test has only 7 mass values vs train's 36; cannot evaluate at
    any other mass value
  - Test contains masses not in train: 296 g and 947 g have no exact
    train counterpart (closest train values: 280, 329 and 926, 940)
  - Test clusters span 3 imaging sessions; train likely concentrated
    in only one (2025-07-13)
  - Train and test are likely DIFFERENT physical clusters (proper
    held-out evaluation), but the dates suggest there is also an
    appearance shift on top of the cluster identity shift

Train mean = 0.548 kg
Test  mean = (108*21 + 165*14 + 296*22 + 430*15 + 600*39 + 765*15
            + 947*24) / 150 = 0.522 kg
(test mean is ~26 g lower than train mean, but distributions differ
 greatly in shape — test has all 7 clusters represented while train
 is right-skewed)
```

Cross-Model Comparison — Architecture and Training

	mass_stage4	clean_v1	clean_v1_LF
Architecture:			
Late fusion	YES	NO	YES
Mass head input ch	257	256	257
Mass head MLP shape	4096->128->1	256->64->1	4096->128->1
Mass head conv lyrs	4 + deconv	3	4 + deconv
Pool	AvgPool(4,4)	GAP(1,1)	AvgPool(4,4)
Approx MLP params	~540k	~17k	~540k
Training:			
Script	train.py	train_new.py	train_new.py
Starting weights	segm_tune (detection-pretrained)	base (cold)	base (cold) (no detection of training)
Max iterations	10,000	15,000	15,000
Base LR	2e-5	1e-4	1e-4
LR schedule	constant	warmup+decay	warmup+decay
mass_loss_weight	2.0	1.0	1.0
Style	staged	joint	joint

Both Model 1 and Model 3 share the SAME mass head and SAME late fusion architecture. Differences are purely in training procedure. Model 2 has the simplified architecture and standard training.

Cross-Model Comparison — Validation Metrics

	mass_stage4	clean_v1	clean_v1_LF
Peak val R^2	0.889	0.594	0.757
at iter	9499	9999	9499
Peak val MAE (kg)	0.100	0.210	0.162
at iter	9499	9999	9499
Final val R^2	0.870	0.544	0.737
Final val MAE (kg)	0.112	0.222	0.168
Final val MRE	0.148	0.769	0.444
Train loss reduction	5.2x	180x	180x
Final train loss	0.033	0.002	0.002
Final val loss	0.136	0.207	0.202
Train/val ratio at end	~4x	~96x	~96x
bbox/AP at convergence	91.7	88.0	88.9
segm/AP at convergence	98.6	96.0	96.9

Interpretation:

- Models 2 and 3 share training dynamics: both heavily overfit training (train/val ratio ~100x) but plateau on val without diverging.
- Model 1 has a much tighter train-val coupling (~4x ratio). Its higher val R^2 is not the result of memorizing training more aggressively (it is memorizing LESS); it is the result of better generalization within the val distribution.
- Late fusion (Model 3 vs Model 2) delivers a clear val gain (R^2 +0.16, MAE -23%) at no detection cost. The architectural contribution is real and measurable.

Cross-Model Comparison — Test-set Performance

Overall test metrics (150 images):

	mass_stage4	clean_v1	clean_v1_LF
Overall MAE (kg)	0.107	0.204	0.127
Overall MAPE	19.4%	87.8%	46.9%
Mean signed bias (kg)	-0.026	+0.175	+0.121
Worst error (kg)	0.401	0.550	0.389

Per-mass-group MAPE (test):

GT(kg)	n	mass_stage4	clean_v1	clean_v1_LF
0.108	21	12.3%	322.4%	167.3%
0.165	14	22.8%	135.1%	31.6%
0.296	22	13.7%	95.2%	52.0%
0.430	15	6.9%	52.1%	35.3%
0.600	39	23.4%	35.0%	31.8%
0.765	15	41.5%	5.2%	7.5%
0.947	24	16.3%	8.2%	2.4%

Per-mass-group bias (test, in kg, signed):

GT(kg)	mass_stage4	clean_v1	clean_v1_LF
0.108	+0.009	+0.348	+0.181
0.165	-0.038	+0.223	+0.052
0.296	-0.027	+0.282	+0.154
0.430	+0.003	+0.224	+0.152
0.600	+0.140	+0.210	+0.191
0.765	-0.317	-0.009	+0.033
0.947	-0.154	-0.078	-0.001

Notable patterns:

- mass_stage4 distinguishes small from large physically (near-zero bias on small clusters).
- clean_v1 and clean_v1_LF have uniformly positive bias on all clusters <0.7 kg (regression toward training-density region).
- mass_stage4 underestimates large clusters; the others do not.
- clean_v1_LF inherits clean_v1's bias pattern but with smaller magnitudes; the LF architecture mitigates but does not cure the distribution-bias failure mode.

Conclusions and Recommendations

Empirical findings:

1. Late fusion architecture has measurable benefit.
   Going from no-LF (Model 2) to LF (Model 3) with identical
   training: val R^2 +0.16, test MAPE -41 percentage points.
   This is a clear win attributable to architecture.

2. Training procedure matters more than architecture for OOD.
   Models 1 and 3 share the SAME architecture. Yet Model 1 has
   test MAPE 19% vs Model 3's 47%. The difference is entirely in:
     - Staged training from a detection-pretrained checkpoint
     - Low LR (2e-5 vs 1e-4)
     - Higher mass_loss_weight (2.0 vs 1.0)

3. The dominant failure mode is regression to training density.
   Joint-from-scratch training (clean_v1, clean_v1_LF) learns the
   density structure of the training mass distribution and
   defaults to it for unfamiliar inputs. Predictions for test
   clusters below the training-density peak are uniformly biased
   upward. Late fusion reduces but does not eliminate this.

4. mass_stage4's behavior is qualitatively different.
   Its predictions track physical cluster size on most groups
   (near-zero bias on 0.108, 0.165, 0.296, 0.430 kg). It fails
   only on one specific cluster (0.765 kg) with high prediction
   variance. This suggests it learned a closer-to-physical
   mapping rather than memorizing training-density priors.

5. Train-val gap is a poor proxy for OOD generalization.
   mass_stage4 (~4x gap) has the best OOD test performance.
   clean_v1_LF (~96x gap) and clean_v1 (~96x gap) generalize
   worse, even though clean_v1 has the same gap as clean_v1_LF.
   The relevant signal is the bias direction on per-cluster test
   performance, not the train-val loss ratio.

Implications for designing a clean mass head:

A. Ensure the mass head has explicit access to depth information
   (late fusion or equivalent). Implicit depth via channel-4 of
   conv1 is empirically insufficient given the RGB-pretrained
   backbone bias.

B. Either train in stages (detection first, then mass) or use a
   loss/LR schedule that gives the mass head time to learn a
   physically grounded mapping before it can overfit to
   training-density priors.

C. Consider injecting EXPLICIT geometric features (computed from
   the predicted mask + depth + camera intrinsics) at the mass
   head's MLP, alongside the visual features. Physical features
   like area_mm^2 and mean_depth_mm cannot be 'pulled toward the
   training mean'; they anchor the prediction in physics.

D. Evaluate on per-cluster bias, not just aggregate R^2 or MAE.
   A model with high val R^2 can still be useless on OOD data
   if its bias varies systematically with cluster size.

E. The training distribution itself is a bottleneck. Without
   adding more diverse small-cluster data, no architectural
   change will fully eliminate the small-cluster prediction error.